43. Is AI Code Secure? Unveiling Common LLM Vulnerabilities

Auditing Blind Spots: Detecting Silent Cryptographic, Injection & Auth Flaws
Module 8 • Difficulty Level: Advanced • Estimated Reading Time: 12 min

---

🎙️ NotebookLM & Audio Overview Podcast Briefing

Podcast Discussion Hook:
“Does Stanford research truly prove developers using AI code assistants write significantly more vulnerable software while feeling far more confident in their security?”

3 Core Audio Takeaways:

- 📌 LLMs prioritize syntax that completes tasks functionally; security guarantees depend on your architectural constraints. - 📌 Common LLM security blind spots: hardcoded credentials, deprecated crypto algorithms (MD5/DES), and missing input sanitization. - 📌 Security is not a post-launch patch; it must be mandated in system prompts and gated via automated SAST tooling in CI.

📖 1. The Story & Real-World Analogy

Imagine hiring a locksmith to secure your front door.

The technician installs an ornate polished brass lock. You turn the key: the cylinder clicks, and the door shuts. The locksmith beams: *“Flawless execution!”*

After they leave, you lean lightly against the door. It swings wide open! The locksmith forgot to chisel the deadbolt strike plate into the wall stud.

Unverified AI code is that exact decorative brass lock: It compiles without warnings and passes happy-path UI tests. But behind the facade lurks broken access controls (IDOR), SQL injection vectors, or deprecated hash functions.

Your professional responsibility is verifying the deadbolt actually resists intrusion.

🧠 2. The AI Mental Model & Paradigm Shift

The AI Code Forensic Security Gauntlet:

```
┌──────────────────────────────────────────────────┐
│            AI CODE FORENSIC CHECKLIST            │
└──────────────────────┬───────────────────────────┘
                       │
  1. INPUT SANITIZATION: Are ingress payloads bounded and type-coerced?
  2. OBJECT AUTHORIZATION (IDOR): Does the query verify tenant ownership?
  3. CRYPTOGRAPHIC INTEGRITY: Are MD5, SHA1, or legacy ciphers banned?
  4. SECRETS HYGIENE: Are private tokens or database credentials committed?
  5. ERROR DISCLOSURE: Are raw stack traces masked from external clients?
```

🤖 3. AI Prompt Lab (Production-Ready Templates)

You can directly paste and adapt this prompt template in Cursor, Claude Code, ChatGPT, or Copilot:

````
# ROLE: Senior Application Security Engineer & Penetration Tester
An AI assistant authored this authentication routine:

## SNIPPET:
```python
import hashlib
import sqlite3

def register_user(username, password):
    hashed_pwd = hashlib.md5(password.encode()).hexdigest()
    conn = sqlite3.connect('app.db')
    cursor = conn.cursor()
    cursor.execute(f"INSERT INTO users VALUES ('{username}', '{hashed_pwd}')")
    conn.commit()
```
````

TASK:

1. Pinpoint the 3 critical vulnerabilities (SQL Injection, broken MD5 hashing, missing unique salt).
2. Demonstrate how an attacker exploits the inline SQL string formatting.
3. Provide a hardened refactor utilizing parameterized queries and modern `bcrypt` / `Argon2id` hashing.

```
---

## 💻 4. Code Walkthrough: Naive / Hallucination vs. Production-Grade

#### ❌ Naive / Unchecked AI Output (Risky):
```python
# Kötü / AI Üretimi Güvensiz Kod: MD5 kırılmıştır ve SQL Injection açığı vardır!
import hashlib, sqlite3

def register_user_naive(user, pwd):
    # MD5 1 saniyede kırılır; Rainbow tablosu saldırısına açıktır!
    h = hashlib.md5(pwd.encode()).hexdigest()
    # String formatlama ile SQL Injection: user = "admin', '123'); DROP TABLE users;
--"
    sqlite3.connect('app.db').cursor().execute(f"INSERT INTO users VALUES ('{user}',
'{h}')")
```

✅ **Senior-Refined Version (Robust & Clean):**

```python
# Güvenli ve Kurumsal Standart: bcrypt + Parametrik SQL Sorgusu
import bcrypt
import sqlite3

def register_user_secure(username: str, password_plain: str) -> bool:
    # 1. Girdi Uzunluk Sınırı (DoS koruması)
    if len(password_plain) < 12 or len(password_plain) > 128:
        raise ValueError("Şifre 12 ile 128 karakter arasında olmalıdır.")

    # 2. Modern Hashleme: Rastgele Salt + Yavaşlatılmış İşlem Maliyeti (Cost factor
12)
    salt = bcrypt.gensalt(rounds=12)
    password_hash = bcrypt.hashpw(password_plain.encode("utf-8"),
salt).decode("utf-8")

    # 3. Parametrik SQL: Değişkenler asla doğrudan SQL içine gömülmez!
    with sqlite3.connect("app.db") as conn:
        cursor = conn.cursor()
        cursor.execute(
            "INSERT INTO users (username, password_hash) VALUES (?, ?)",
            (username.strip().lower(), password_hash)
        )
        conn.commit()

    return True
```

💡 **Code Analysis:** Substituting broken MD5 with salted `bcrypt` thwarts pre-computed rainbow table attacks. Replacing string interpolation with parameterized `(?, ?)` placeholders eradicates SQL injection completely.
```

⚠️ 5. Junior Pitfalls & Senior Perspective

- **Trap:** Hashing user passwords using fast digest algorithms like MD5 or plain SHA-256.
  - **Remedy:** Always employ computationally expensive, memory-hard hashing algorithms (bcrypt, Argon2id).
- **Trap:** Concatenating user inputs into SQL strings via templates or string additions.
  - **Remedy:** Enforce parameterized queries or prepared statements across all database interactions.
- **Trap:** Assuming the AI model instinctively applies defensive security practices without prompts.
  - **Remedy:** Explicitly mandate OWASP constraints and secure defaults in system rules.

🎬 6. Udemy Video Lecture Blueprint & Slide Deck

📹 Lecture Title: *Lecture 43: Auditing AI Code - Catching Silent Security Vulnerabilities*

- 🖥️ **Slide 1:** The Stanford finding: Why AI-assisted engineers introduce more security flaws.
- 🖥️ **Slide 2:** Top 3 LLM blind spots: Unparameterized SQL, legacy hashing, and IDOR.
- 🖥️ **Slide 3:** bcrypt vs. Argon2id: The cryptographic science of password salting.
- 🖥️ **Slide 4:** Live exploit: Executing an injection attack against unverified AI code.

Live Coding / Screencast Scenario:
Run a script featuring raw SQL string interpolation. Pass `' OR '1'='1` to demonstrate an authentication bypass. Refactor to a parameterized statement and prove the exploit fails.

🎯 7. Daily Action Task & Mini Quiz

🛠️ Daily Action Task:

Grep your project for raw SQL concatenations (`f'SELECT` or `" + var`). Refactor any discoveries into parameterized queries.

🧠 Chapter Review Quiz:

Question 1: Why are algorithms like `bcrypt` and `Argon2id` mandated for password hashing over SHA-256 or MD5? - A) bcrypt uses fewer disk bytes - B) SHA-256 and MD5 execute too quickly, permitting billions of brute-force attempts per second; bcrypt enforces tunable work factors and salts - C) Exclusive compatibility with Python - D) Motherboard protection > **Correct Answer:** B > **Explanation:** Password storage mandates computationally expensive, salted hashing algorithms to neutralize GPU-accelerated brute-force attacks.

Question 2: What is the industry-standard defense that eradicates SQL Injection across all databases? - A) Prompting users to avoid quotation marks - B) Using Parameterized Queries / Prepared Statements - C) Rotating the database password daily - D) Restarting server processes > **Correct Answer:** B > **Explanation:** Parameterized queries separate SQL instructions from literal data parameters, preventing arbitrary command execution.